



计算机工程与应用
Computer Engineering and Applications
ISSN 1002-8331, CN 11-2127/TP

《计算机工程与应用》网络首发论文

题目: 带宽受限型多核加速器架构的软硬件协同优化方法
作者: 肖星宇, 程虎, 杨赟辉, 魏敬和, 张学永
网络首发日期: 2026-05-19
引用格式: 肖星宇, 程虎, 杨赟辉, 魏敬和, 张学永. 带宽受限型多核加速器架构的软硬件协同优化方法[J/OL]. 计算机工程与应用.
<https://link.cnki.net/urlid/11.2127.TP.20260519.1332.006>



网络首发: 在编辑部工作流程中, 稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定, 且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式(包括网络呈现版式)排版后的稿件, 可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定; 学术研究成果具有创新性、科学性和先进性, 符合编辑部对刊文的录用要求, 不存在学术不端行为及其他侵权行为; 稿件内容应基本符合国家有关书刊编辑、出版的技术标准, 正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性, 录用定稿一经发布, 不得修改论文题目、作者、机构名称和学术内容, 只可基于编辑规范进行少量文字的修改。

出版确认: 纸质期刊编辑部通过与《中国学术期刊(光盘版)》电子杂志社有限公司签约, 在《中国学术期刊(网络版)》出版传播平台上创办与纸质期刊内容一致的网络版, 以单篇或整期出版形式, 在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊(网络版)》是国家新闻出版广电总局批准的网络连续型出版物(ISSN 2096-4188, CN 11-6037/Z), 所以签约期刊的网络版上网络首发论文视为正式出版。

带宽受限型多核加速器架构的软硬件协同优化方法

肖星宇¹, 程虎², 杨赟辉², 魏敬和^{1,2}, 张学永¹⁺

1.东南大学 集成电路学院, 南京 211189

2.中国电子科技集团第五十八研究所, 江苏 无锡 214000

+ 通信作者 E-mail: xueyong.zhang@seu.edu.cn

摘要：深度学习神经网络参数量激增使人工智能加速器面临严峻存储墙挑战。传统计算-缓存紧耦合架构虽通过数据局部性优化缓解了高精度中间数据的带宽压力，却因权重与激活值的高频次片外重复访存形成系统性性能瓶颈。针对上述问题，采用软硬件协同设计方法，通过多核协同广播机制和可重构计算-缓存架构实现本地数据的全局化高效复用以减少片外访问，灵活有效适配多样化网络层的算术强度需求；同时构建算子层动态映射优化框架，将数据流、分块策略与硬件配置协同优化，并引入结构化解空间约简原则，实现片上存储复用效率最大化。最后，基于优化后的多核加速器架构完成了实片性能验证。实验结果表明，所设计的加速器在YOLOv5s推理任务中能效表现优于昇腾310，并达到GPU的20倍以上，优化效果显著。

关键词：加速器；带宽优化；片外存储访问；卷积神经网络；多核

文献标志码：A **中图分类号：**TP368.2 **doi：**10.3778/j.issn.1002-8331.2512-0391

Hardware-Software Co-Optimization Methodology for Bandwidth-Constrained Multi-Core Accelerator Architectures

XIAO Xingyu¹, CHENG Hu², YANG Yunhui², WEI Jinghe^{1,2}, ZHANG Xueyong¹⁺

1.School of Integrated Circuits, Southeast University, Nanjing 211189, China

2.The 58th Research Institute of CETCs, Wuxi, Jiangsu 214000, China

Abstract: The exponential growth of parameters in deep neural networks has imposed severe memory wall challenges on AI accelerators. While conventional computing-cache tightly-coupled architectures alleviate bandwidth pressure for high-precision intermediate data through data locality optimization, they suffer from a systemic performance bottleneck due to frequent off-chip repeated accesses of weights and activations. To address these issues, a hardware-software co-design methodology is adopted, utilizing a multi-core collaborative broadcast mechanism and a reconfigurable computing-cache architecture to achieve efficient global reuse of local data, thereby reducing off-chip memory access and flexibly adapting to the diverse arithmetic intensity requirements of various network layers. Additionally, a dynamic mapping optimization framework at the operator level is established, which co-optimizes dataflow, tiling strategies, and hardware configurations. By incorporating structured design space pruning principles, the framework maximizes on-chip storage reuse efficiency. Finally, a physical chip based on the optimized multi-core accelerator architecture is implemented for performance validation. Experimental results demonstrate that the proposed accelerator achieves superior energy efficiency in YOLOv5s inference tasks compared to the Ascend 310, and outperforms a comparable GPU by over 20 times, confirming the significant effectiveness of the proposed optimizations.

Key words: accelerator; bandwidth optimization; external memory access; CNN; multi-core

基金项目：集成电路与微系统国家重点实验室-集团稳定支持项目(JCYQ2500401)。

作者简介：肖星宇(2001—)，男，硕士研究生，CCF 学生会员，研究方向为边缘端人工智能推理加速器架构、高性能计算；程虎(1989—)，男，博士，工程师，研究方向为机器视觉、智能计算、高性能计算；杨赟辉(1994—)，男，博士研究生，工程师，研究方向为机器视觉、智能计算、高性能计算；魏敬和(1970—)，男，博士，研究员，研究方向为系统芯片；张学永(1990—)，男，博士，教授，研究方向为数模混合芯片设计、硬件加速与类脑计算等。

深度学习技术的快速发展,大幅提升了人工智能应用在图像分类、目标检测^[1-2]及自然语言处理^[3]等关键领域的性能边界,实现了前所未有的精度水平。作为AI核心引擎的神经网络模型,在算法持续优化与应用场景不断拓展的双重驱动下,其识别能力持续增强,同时模型深度与结构复杂度也显著提升^[4]。

如今,单次推理过程所需参数量常达数十兆至数百兆量级^[5],这带来了庞大的数据交互需求与频繁的片外存储器访问,对系统吞吐率与并行处理能力构成严峻挑战。伴随阵列处理器中集成核心数量的增长,算力得以提升,但处理器与存储器之间的性能鸿沟也日益凸显。面对神经网络等兼具数据密集与计算密集特征的人工智能负载,持续提升计算并行度^[6-7]以加速算法执行变得至关重要,这对系统带宽与算力提出了更高的双重需求。针对这一挑战,主流硬件加速架构普遍采用输入数据低精度量化策略^[8],通过降低存储位宽来实现存储资源优化和带宽压力缓解。

然而,为满足模型精度要求,卷积/矩阵乘法的中间计算结果仍需保持全精度累加运算,这种混合精度计算模式导致高精度中间结果的数据传输成为新的性能瓶颈。为应对这一挑战,当前先进加速器架构普遍采用计算单元紧耦合的累加缓存单元设计策略^[9]。该技术通过将累加操作限制在计算阵列的局部缓存区域,有效减少了高精度中间数据在存储层次结构中的迁移次数。然而,现有累加缓存机制在片上缓存资源受限的情况下,当模型参数量超过缓存容量阈值时,激活值和权重在片外存储结构中的反复访存成为了性能瓶颈。尽管已有研究通过数据重用^[10-13]、压缩编码^[14]等方式优化访存效率,但面向多核并行架构的系统级带宽优化机制仍存在理论模型缺失与设计空间探索不足的问题。

为突破上述带宽瓶颈,本文系统性地提出了一种基于张量计算数据特征挖掘的多粒度协同带宽优化方案。该方案构建了一个融合多核协同广播机制、可重构计算-缓存架构与算子层动态映射优化框架的系统的带宽优化策略体系。该体系能够深度感知数据依赖性与局部性特征,智能地在多核间分配计算任务,并动态优化单数据块形状与架构配置。这种协同优化机制显著提升了片上存储层次的数据局部性利用率,高效协调了计算与访存操作,从而显著缓解了片外存储器的访问带宽压力。实验结果表明,该方案针对带宽受限型算子有效降低了系统对访存带宽的需求,最终实现了张量计算性能的显著提升。本文创新点和贡献有4个方面:

1)设计了可重构计算-缓存架构。将算力与缓存资源从核级静态绑定中解耦,使系统能够在任务粒度动态调整算力-缓存配比,灵活适配多样化网络层的算术强度需求,为软件映射策略释放了结构性的硬件灵活性。

2)揭示并行策略与层特征的动态适配规律。基于系统实验分析表明,最优并行策略随网络深度及张量维度演化而动态变化,论证了自适应部署的必要性,为多核并行维度的选择提供了量化依据。

3)构建算子层动态映射优化框架。将数据流、分块策略与硬件配置纳入协同优化范畴,提出最大分块、整分与方形分块三项结构化解空间约简原则,在保证优化质量的前提下显著降低搜索复杂度,使框架具备面向CNN、Transformer等多样化模型的普适性与高效性。

4)基于上述提出的优化方法,本文设计了一种NPU架构进行实片实验验证。实验结果显示,所设计的加速器在YOLOv5s推理任务中能效表现优于昇腾310,并达到GPU的20倍以上。

1 相关背景与工作

当前主流的神经网络等AI负载兼具高度数据密集与计算密集的特征,对算力提出了持续增长的严苛需求。为应对这一挑战,深度神经网络加速器作为人工智能计算系统的核心执行单元,其架构已显著演进为多核系统范式^[15]。其核心计算引擎通常采用基于复杂片上互连网络的处理单元阵列(processing element array, PEA)。每个处理单元集成了本地缓冲区和专用的算术逻辑单元,负责处理基础数据操作。全局缓冲区和权重缓冲区则紧邻PEA部署,分别用于缓存激活值和权重参数。它们不仅充当PEA与片外存储器之间的关键数据接口,更负责管理两者间高带宽的数据传输调度。

然而,受限于芯片面积与功耗的约束,全局缓冲区的容量通常无法充分满足整个网络层的即时数据需求。因此,编译器必须将完整的网络执行过程细粒度地划分为一系列可管理的基本计算任务。这些任务需在PEA提供的空间并行资源和时间维度上进行精密的调度编排^[16-18]。在此过程中,全局缓冲区的有限容量成为主导片外数据访存频率和带宽需求的关键因素,进而对加速器的整体性能和能效比产生决定性影响。正是这一片外访存(external memory access, EMA)瓶颈,构成了当前深度神经网络加速器性能提升的主要制约。

为应对深度神经网络加速器面临的数据传输瓶颈,当前研究工作主要聚焦于优化张量数据结构和优化内存访问关系两大方向。

张量数据结构优化核心策略在于通过稀疏化^[19-21]与量化^[22-24]技术,显著压缩神经网络权重与激活张量。稀疏化利用模型固有的冗余性,识别并剔除接近零的无效数据元素;量化则通过降低数据精度,减小单个数据元素的存储空间。这两种手段协同作用,有效降低了片外存储与片内缓存的数据传输量及存储开销。

内存访问关系优化主要通过缓存架构与PEA布局协同优化和基于数据重用的内存数据映射优化两种

途径减少能耗和数据传输。

存内计算 (processing-in-memory, PIM) [25-31] 是一种突破性的架构范式, 其核心思想是将计算单元嵌入存储器内部或紧邻存储器放置, 直接在数据存储位置执行计算操作。PIM 技术尤其适用于数据局部性差、缓存命中率低的计算密集型应用场景。在此类场景中, PIM 能够显著消除传统冯·诺依曼架构中数据在处理器与存储器间频繁搬运的开销, 从而有效降低由数据传输主导的高额系统能耗。然而, 现有存算一体架构主要聚焦于优化片内的数据移动, 对于大规模的片外数据访存缓解能力仍然有限。换言之, PIM 虽能优化片内数据流能效, 但尚未从根本上解决由全局缓冲区容量限制引发的片外访存瓶颈问题。

数据重用是缓解片外访存压力的核心策略之一。寒武纪公司的 DianNao 架构[10]首次系统性提出并验证了利用神经网络张量运算中固有的强数据共享特性进行片内高效重用的价值。通过在计算核心内部或邻近存储层次中复用已加载的数据, 可显著避免对片外存

储器的冗余访问。传统的分布式存储架构不利于跨单元的数据共享。为克服此局限, 谷歌的 TPU[13]创新性地采用了统一缓存设计, 将指令和激活数据/部分权重整合存储在单一、共享的片上缓存中。这种设计极大地简化了缓存一致性管理, 降低了多核间数据共享的访问延迟与复杂度, 为高效的数据重用提供了关键的硬件基础。

神经网络加速器执行任务的粒度主要分为层粒度和图粒度, 其选择受到片上缓冲区容量和数据流特性的共同制约。基于数据重用的内存数据映射优化, 其核心在于编译器在硬件映射阶段, 通过智能规划层粒度或图粒度的计算任务, 以最大化激活值和权重的片内复用机会, 从而大幅降低片外访存的频率和带宽需求。然而, 现有基于数据重用的映射优化方法[32-37]普遍存在通用性不足的问题。其优化效果过度趋向于某一特定神经网络模型的结构特性, 难以在不同类型或复杂度的模型上实现一致且显著的性能提升。这种局限性限制了现有方案在实际部署场景中的广泛应用。

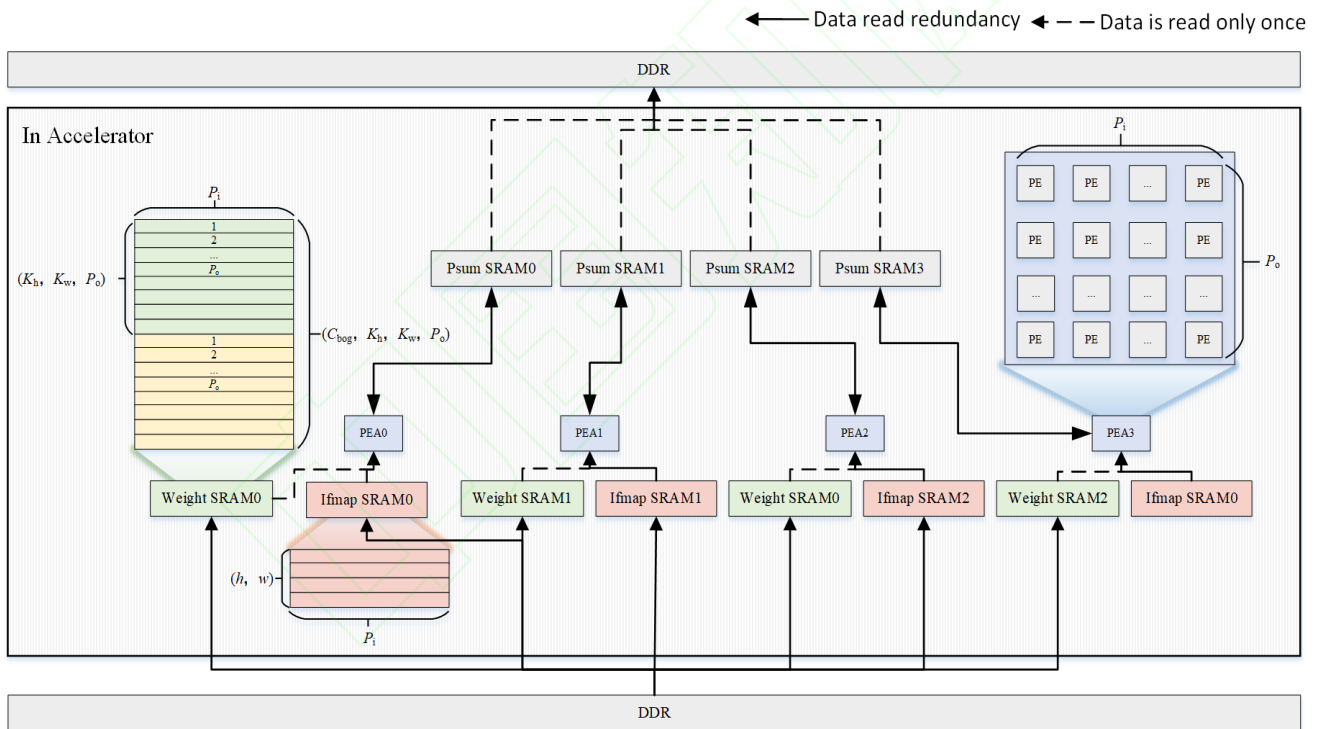


图1 多核 OS 数据流数据访存

Fig.1 Multi-core OS Data Flow Data Access

2 多核张量计算数据流与空间并行

在现代神经网络加速器中, 多核架构被广泛用于提升张量计算的吞吐量与能效。基于累加缓存机制的加速器设计, 通常采用输出固定 (output stationary, OS) 的数据流调度方式, 以降低片外交互频率。如图 1 所示, 在 OS 数据流中, 输出张量在整个计算过程中仅需写回片外存储器一次, 从而显著减少对外部带宽的占

用, 提升系统能效。输入激活值与权重的访存行为则呈现不同的复用特征: 其片内复用率取决于在相应计算维度上可被缓存的容量与总数据量之间的比例关系。具体而言, OS 数据流通过将输出通道维度的部分和保留在片上累加器中, 实现了输出数据的本地化更新, 避免了频繁的片外读写。

为进一步提升输入激活值与权重的复用效率, 计算循环需在时间和空间维度上进行合理的展开与映射。

如图 2 所示, OS 数据流通常采用多维循环展开策略: 在空间维度上, 将输出通道数 C_{out} 与输出特征图的空间尺寸 $H'W'$ 在多个计算核之间并行展开; 在时间维度上, 则将输入通道数 C_{in} 及卷积核尺寸 K_h 、 K_w 对应的循环依次展开, 使得每个计算核能够在连续的时间步内处理一组固定的输入激活值与权重。这种空间并行加时

间流水的调度方式, 不仅提升了数据复用率, 还有效隐藏了片外访存延迟。Transformer 架构中的矩阵张量运算可用卷积核尺寸为 1 的卷积运算表示, 上述数据流策略同样适用于其加速实现。表 1 总结了图 1 与图 2 中所涉及的主要参数, 包括输入输出通道数、特征图尺寸、卷积核尺寸以及计算核的并行度等, 为后续的性能建模与资源评估提供了统一的描述基础。

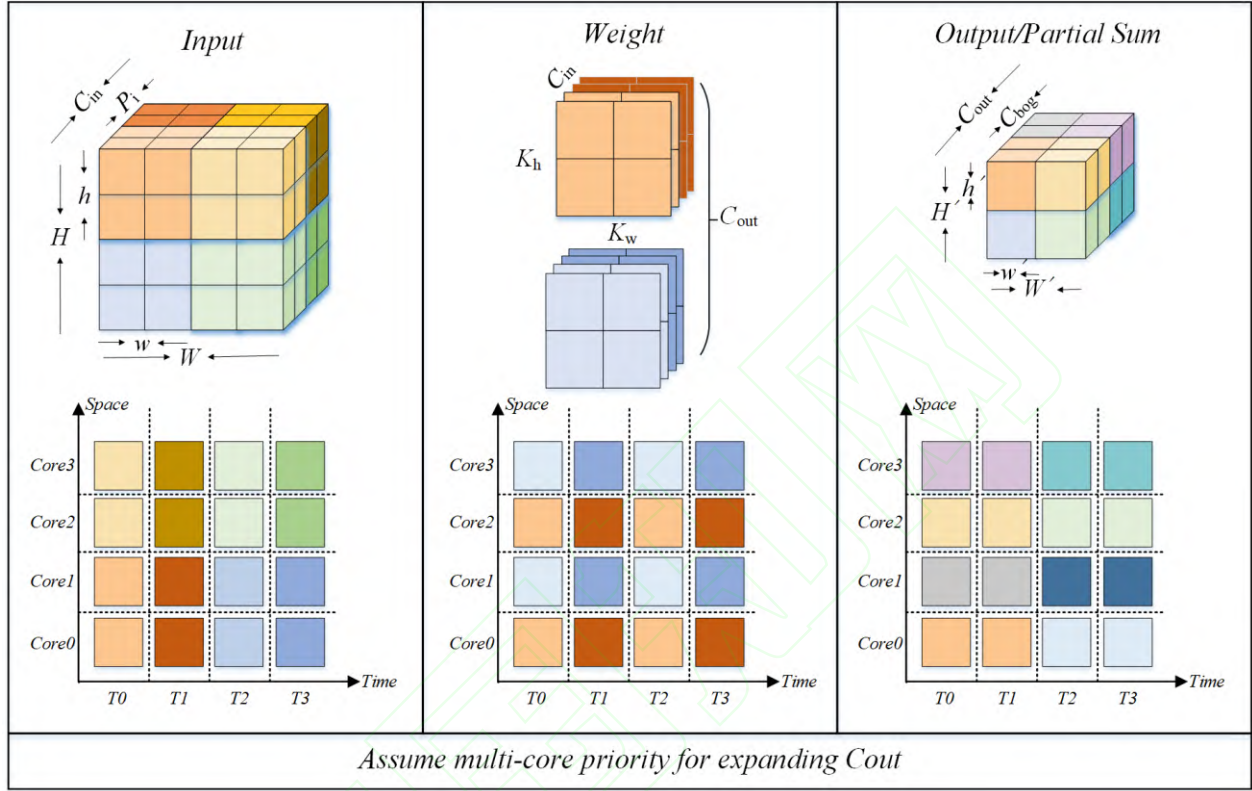


图 2 OS 数据流的时间展开与空间并行

Fig.2 Temporal Unfolding and Spatial Parallelism in OS Data Flow

表 1 参数介绍

Table 1 Parameter Introduction

参数	介绍
P_o	PEA 输出通道计算并行度
P_i	PEA 输入通道计算并行度
C_{in}	输入通道
C_{out}	输出通道
C_{beg}	输出特征图/部分和分块的输出通道组数 (P_o 为一组)
H, W	输入特征图的高和宽
h, w	输入特征图分块的高和宽
H', W'	输出特征图/部分和的高和宽
h', w'	输出特征图/部分和分块的高和宽

K_h, K_w

卷积核的高和宽

3 多核架构带宽优化策略

基于前述数据流结构, 本文提出一套系统的带宽优化策略体系。首先, 通过多核协同广播传输机制实现激活值与权重的核间共享, 从数据流层面提升复用效率; 在此基础上, 将算力与缓存资源从核级静态绑定中解耦, 提出可重构计算-缓存架构, 使系统能够在任务粒度动态调整算力-缓存配比以适配多样化层特征。结合典型模型的分析揭示, 最优并行策略随层特征演化而动态变化, 论证了自适应部署的必要性。进而构建算子层动态映射优化框架, 将数据流、分块策略与硬件配置纳入协同优化范畴, 并引入结构化解空间约简策略以降低搜索复杂度。上述策略共同构建了一个兼具数据复用效率、资源弹性适配与映射灵活性的多核加速器带宽优化体系, 为架构设计与编译映射提供了理论

依据。

3.1 多核协同广播传输技术

上文已经提到，在既定的累加缓存机制架构下，激活值和权重不可避免的产生重复加载的行为。基于上述数据流，其数据复用特性可通过以下公式表征：

对于卷积运算：

传统单核架构输入特征图 (Ifmap) 在空间维度 (HW) 的重加载系数 α ：

$$\alpha = \left\lceil \frac{C_{out}}{C_{bog} P_o} \right\rceil \quad (1)$$

传统单核架构权重 (Weight) 在输出通道维度 (C_{out}) 的重加载系数 β ：

$$\beta = \left\lceil \frac{H'W'}{h'w'} \right\rceil \quad (2)$$

重加载系数表征两类数据因缓存容量不足导致的重复访存次数，该关系表明数据重复加载次数与参

数的分块策略密切相关，而传统单核架构受限于存储资源约束，存在以下优化悖论：

$$C_{bog} P_o h'w' \leq S_{cache} \quad (3)$$

公式(3)中 S_{cache} 为累加缓存容量。该不等式揭示了存储-并行度的固有矛盾：在 S_{cache} 固定约束下， $C_{bog} P_o$ (输出通道并行度) 与 $h'w'$ (空间分块尺寸) 呈反比关系，二者无法同时取最大值。传统单核架构因此陷入优化悖论——提升通道并行度必然压缩空间分块，反之亦然。

本研究提出的多核协同架构突破该限制，通过图3所示的广播传输机制实现：将 $h'w'$ 或 $C_{bog} P_o$ 子块通过片上传输网络广播至所有计算核所对应的缓存中，实现多核间激活值或权重的共享。同时，通过核间协作的方式将 $C_{bog} P_o / h'w'$ 扩展为 $n C_{bog} P_o / n h'w'$ ， n 为激活的计算核心数量。

该技术可使权重和激活值的复用率同时得到提升，大大减少了多核加速器带宽需求。

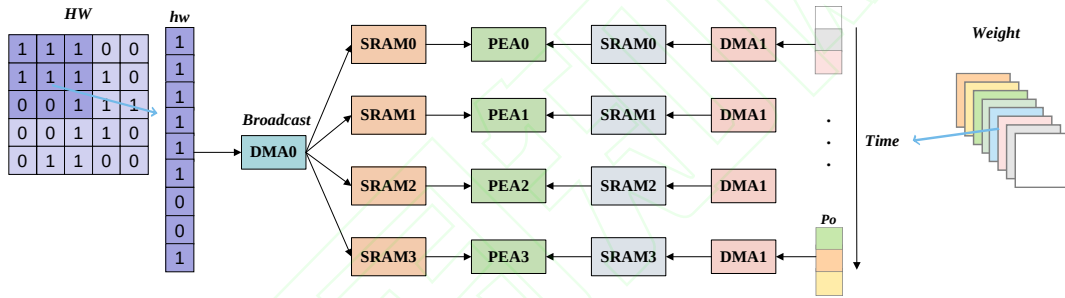
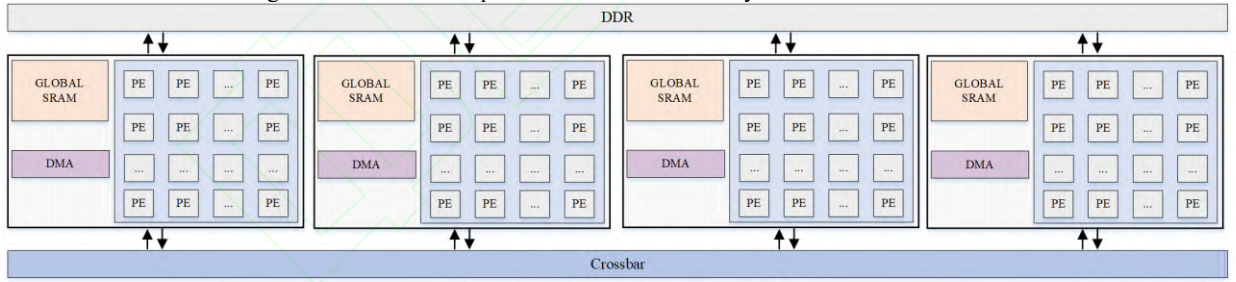
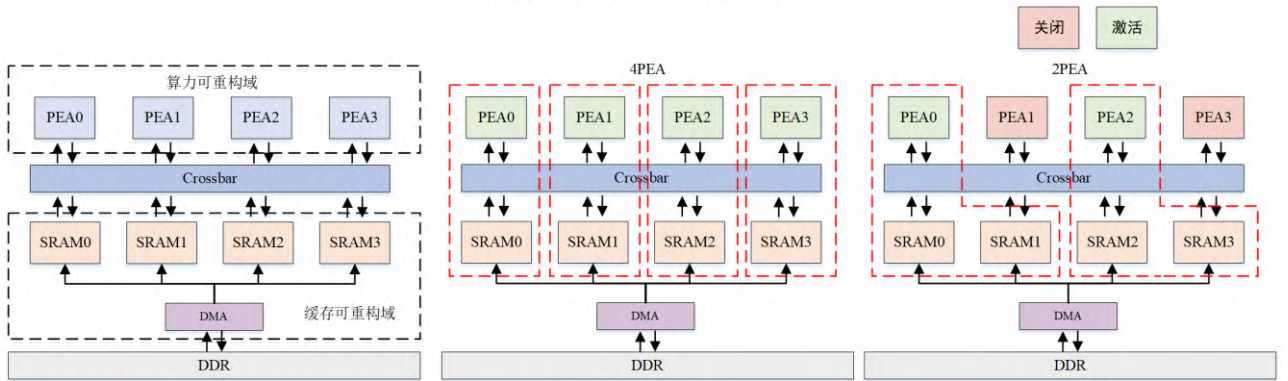


图3 多核协同广播传输机制

Fig.3 Multi-Core Cooperative Broadcast Delivery Mechanism Schematic



(a)传统核级资源静态绑定架构



(b)可重构算力-缓存架构

图4 传统架构与可重构算力-缓存架构

Fig.4 Traditional Architecture vs. Reconfigurable Computing-Cache Architecture

3.2 多核可重构计算-缓存架构

算术强度定义为模型层所需总算力与总带宽之比,而给定网络层的最优硬件配置,应使其算力-带宽比恰好等于该层的算术强度。当前,基于 ASIC 的加速器架构探索大多聚焦于面向特定模型寻求最优的架构参数与存储配置。然而,ASIC 芯片的研发周期远滞后于模型更新速度,这一矛盾对加速器的可重构能力提出了更高要求。与此同时,传统多核加速器普遍采用“核级资源静态绑定”的组织模式(图 4(a)),导致片上缓存资源难以根据任务特征进行集中调度与动态重组,严重限制了架构的软件可重构性。基于此,本文提出一种核级资源角色(计算-缓存)可重构架构,如图 4(b)所示,该架构通过将计算能力与缓存资源从固定的核级绑定关系中解耦,使系统能够在任务级别动态调整算力规模与有效缓存容量之间的比例关系,从而更灵活地适配多样化网络层对算力-带宽比的需求。具体来说,该架构将多核加速器中的算力资源与缓存资源重新定义为两类可独立配置的逻辑角色。计算核提供激活的计算阵列,缓存供给核仅保留缓存功能并关闭计算阵列。通过对两类核心数量的联合配置,系统可在不增加硬件规模的前提下,形成可调节的算力-缓存比例。当任务特征表现为带宽受限时,系统可通过减少计算核数量、增加缓存供给核数量,换取更大的有效片上缓存容量,从而降低对片外 DRAM 的访问压力;反之,在算力主导的场景下,系统可增加计算核数量以提升并行度。这一设计在硬件实现上保持每核私有 SRAM 的物理结构不变,仅通过任务级配置建立或撤销跨核缓存访问关系,既控制了实现复杂度,也为数据复用效率的提升提供了结构性的硬件基础。

PINGPONG	CORE SELECT	SRAM DEPTH
----------	-------------	------------

图 5 地址编码

Fig.5 Address Encoding

如图 5 所示,架构采用显式地址编码的跨核读写机制,将共享语义直接封装于地址字段,实现多核请求的统一解码、路由与时序隔离。地址编码由三部分构成:低位字段指定目标 SRAM 内的存储位置;中位字段标识目标核缓存,在单核地址空间用满后可自动跳变至下一核缓存;高位字段用于乒乓相位选择,显式区分当前计算读缓冲区与下一阶段装载写缓冲区。硬件对该编码解码后生成对应互联路由与读/写控制信号。通过将“存储落点-核目标选择-时序相位”在地址层面显式化,该机制将跨核共享的可见性与并发边界收敛为清晰的硬件规则,从而显著降低实现与验证的复杂度。

本文提出的可重构计算-缓存架构,通过将算力与缓存资源解耦,实现了任务级别的动态重组,从而为软

件映射策略的设计释放了前所未有的灵活性。如图 6 所示,架构通过动态调整算力规模与有效缓存容量之间的配比,映射算法得以突破静态资源边界的约束,依据计算任务的访存特征与并行度需求,在更广阔的搜索空间内寻求最优解。这种映射灵活性的提升,使得硬件平台能够更精细地适配多样化的网络层行为,优化计算-通信重叠效率,并有效减少片外访存开销。

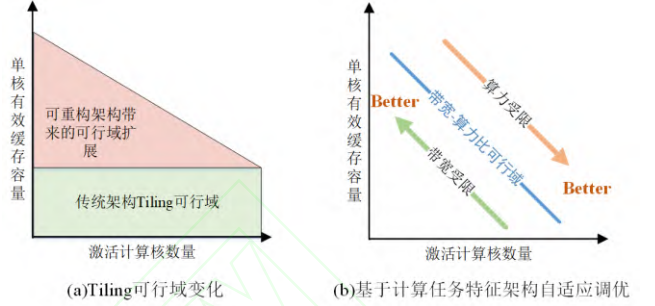


图 6 Tiling 可行域变化与基于计算任务特征架构自适应调优

Fig.6 Tiling Feasible Domain Variation and Architecture Adaptive Tuning Based on Computational Task Characteristics

3.3 多核并行策略

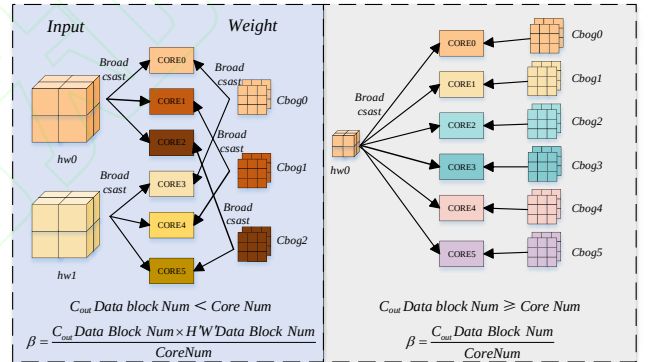
图 7 OSC 重加载系数 β 变化

Fig.7 OSC Re-loading Coefficient Variation

基于 3.1 节建立的公式,多核优先部署的维度可形式化描述为二维决策空间 $D = \{C_{out}, H'W'\}$ 。将优先部署 C_{out} 维度的并行策略记为 OSC,优先部署 $H'W'$ 维度的并行策略记为 OST。与单核架构不同,在多核架构的混合并行框架下,激活值和权重的重加载系数 α 和 β ,需根据优先部署维度的分块数相对于可用计算核心数的关系进行动态决策。图 7 是以 OSC 为例展示优先部署维度所对应的输入数据(OSC 为权重,OST 为激活值)重加载系数变化。OSC 和 OST 的重加载系数表达式分别如式(4)和(5)所示。

$$\alpha = \left\lceil \frac{C_{out}}{nC_{bog}P_o} \right\rceil, \beta = \min \left(\left\lceil \frac{H'W'}{h'w'} \right\rceil, \left\lceil \frac{C_{out}H'W'}{nC_{bog}h'w'P_o} \right\rceil \right) \quad (4)$$

$$\alpha = \min \left(\left\lceil \frac{C_{out}}{C_{bog}P_o} \right\rceil, \left\lceil \frac{C_{out}H'W'}{nC_{bog}h'w'P_o} \right\rceil \right), \beta = \left\lceil \frac{H'W'}{nh'w'} \right\rceil \quad (5)$$

基于上述重加载系数,数据的带宽需求可由重加载系数与原数据量的乘积获得,故对于一个卷积/矩阵乘张量计算的片外输入数据总带宽如下,其中 b_{act} 为数据精度位宽:

$$BW = (\alpha C_{in} HW + \beta C_{out} C_{in} K_h K_w) b_{act} \quad (6)$$

上述数据带宽公式均建立在片上缓存容量无法完整容纳全部输入激活值或权重数据的预设条件下。若缓存具备完全装载单类数据的能力,则相应数据仅需从片外存储器读取一次,无需重复访存,其总带宽可简化为公式(7)所示:

$$BW = (C_{in} HW + C_{out} C_{in} K_h K_w) b_{act} \quad (7)$$

为系统评估多核部署对数据带宽的影响,并确保分析结论对不同神经网络架构的普适性,本研究选取两种代表性的模型作为基准:基于 Transformer 的视觉模型 ViT-Base 与典型卷积神经网络 ResNet-18。实验在统一硬件配置下进行,权重与激活精度均为 FP16。ViT-Base 采用标准编码器结构,输入序列长度为 197,共包含 12 层结构相同的编码器,因此其单层参数可反映整体计算特征。ResNet-18 为标准残差网络,输入分辨率为 224×224 ,其主体由四个阶段的残差块构成,各阶段在通道数与空间维度上具有显著差异,导致层间计算特征不一致。为全面评估其通信行为,本研究将对 ResNet-18 中所有卷积层及张量变换层的片外数据通信量进行完整测试与分析。

为优化访存性能,本文在实验开展前对单核内存配置进行了合理性的分析与探索。对于单核,将公式(1)、(2)的 α 和 β 代入公式(6)中可得到式(8)所示的总带宽表达式。

$$BW = C_{out} C_{in} \left(\frac{HW}{C_{bog} P_o} + \frac{H'W'K_h K_w}{h'w'} \right) \quad (8)$$

据此,可构建如式(9)所示的约束优化问题,其目标为最小化片外数据带宽。约束条件规定如下:约束一要求输出张量分块尺寸 $C_{bog} h'w' P_o$ 小于片上累加缓存容量 S_{cache} 与输出张量总尺寸;约束二要求激活值分块尺寸 $hw P_i$ 小于激活值缓存容量 I_{cache} 与激活值总尺寸;约束三要求权重分块尺寸 $C_{bog} P_o P_i K_h K_w$ 小于权重缓存容量 W_{cache} 与权重总尺寸。

$$\begin{aligned} h', w', C_{bog} &= \arg \min_{h', w', C_{bog}} \left(\frac{HW}{C_{bog} P_o} + \frac{H'W'K_h K_w}{h'w'} \right) \\ \text{s.t. } C_{bog} h'w' &\leq \frac{\min\{S_{cache}, C_{out} HW\}}{P_o} \\ hw &\leq \frac{\min\{I_{cache}, C_{in} HW\}}{P_i} \\ C_{bog} &\leq \frac{\min\{W_{cache}, C_{out} C_{in} K_h K_w\}}{P_o P_i K_h K_w} \\ C_{bog}, h', w' &\in \mathbb{Z}_0^+ \end{aligned} \quad (9)$$

上述模型在约束条件 $C_{bog} h'w' \leq \frac{S_{cache}}{P_o}$ 下,根据算

术-几何平均不等式可得当约束目标中相加的两项相等时,取得理论最小值。基于此,可联合约束条件求得缓存深度最优比例,其中 $stride$ 为步长:

$$\begin{cases} I_{cache} = hw P_i \\ W_{cache} = C_{bog} P_o P_i K_h K_w \\ \frac{HW}{C_{bog} P_o} = \frac{H'W'K_h K_w}{h'w'} \Rightarrow I_{cache} = W_{cache} \\ hw \approx stride^2 h'w' \\ HW \approx stride^2 H'W' \end{cases} \quad (10)$$

由上述分析可知,在对称缓存配置下可获得理论带宽下界,故而实验中对激活值缓存与权重缓存分配相同容量。

基于上述实验配置,本文完成了对 ViT-Base 和 ResNet-18 模型的片外数据通信量测试。如图 8 所示,Transformer 架构模型 ViT-Base 在 OSC 和 OST 两种并行策略下输入数据总带宽并无变化。表 2 给出 ViT-base 部分层尺寸,表中激活值/权重的数据结构为 $(H, W, C_{in}, C_{out}) / (C_{out}, C_{in}, K_h, K_w)$ 。观察表 2 可知,Transformer 模型的层参数特征可归为两类。第一类为多头注意力分离计算层(如 Layer4、6),其各张量尺寸较小,易于完整缓存于片上存储中,使得 OSC 与 OST 两种部署模式的带宽性能均逼近理论下限,结果趋于一致。第二类为多头合并张量计算层(如 Layer1~3、7、9~10),其隐层维度 Dim 较大,片上缓存难以完整容纳任一类型张量,访存性能因而受到显著约束。对于第二类情形,因 Dim 较大,优先部署维度下的最优数据分块尺度通常超出核的数量。将此情形下的 α 和 β 代入式(6),并考虑 $K_h = K_w = 1$ 、 $HW = H'W'$ 以及 $Dim = C_{out} = C_{in}$,则可得:

$$BW_{OSC} = Dim^2 HW \left(\frac{1}{nC_{bog} P_o} + \frac{1}{h'w'} \right) \quad (11)$$

$$BW_{OST} = Dim^2 HW \left(\frac{1}{C_{bog} P_o} + \frac{1}{nh'w'} \right) \quad (12)$$

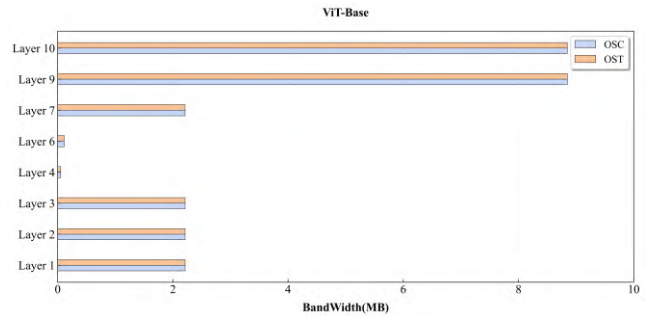


图 8 ViT-Base 在不同并行策略下的输入数据总带宽

Fig.8 Total Input Data Bandwidth of ViT-Base Under Different Parallel Strategies

由于 Dim 、 H 以及 W 皆为常数,令 $C_{bog} P_o$ 为 x ,

$h'w'$ 为 y , 在约束条件 $x \leq \frac{W_{cache}}{P_i}$, $y \leq \frac{I_{cache}}{P_i}$, $xy \leq \frac{S_{cache}}{P_o}$ 且对称缓存配置 $W_{cache} = I_{cache}$ 下, 有:

$$BW_{OSC} = \frac{1}{nx} + \frac{1}{y} = BW_{OST}(y, x, n) \quad (13)$$

这一对偶对称性表明: 对于 Transformer 这类模型来说, 两种策略的带宽性能在数学上等价, 解释了图 8 中 OSC 与 OST 柱状图重合的实验现象。

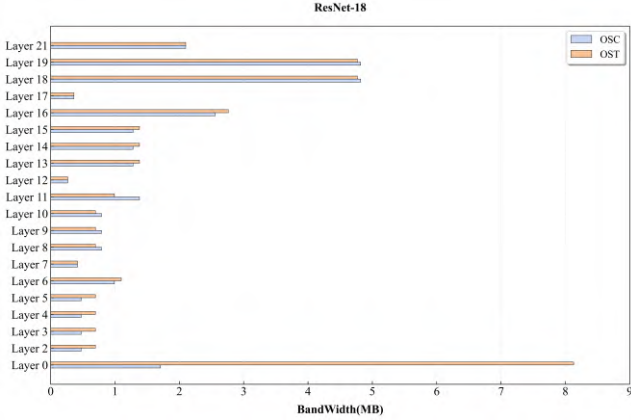


图 9 ResNet-18 在不同并行策略下的输入数据总带宽

Fig.9 Total Input Data Bandwidth of ResNet-18 Under Different Parallel Strategies

如图 9 所示, 面向卷积神经网络 ResNet-18, OSC 与 OST 两种并行部署策略在数据带宽上呈现显著差异, 最大差异可达 79.01%。

表 3 给出 ResNet-18 模型部分层尺寸。在 ResNet-18 模型的浅层部分 (Layer0~6), 相较于 OST 部署策略, OSC 策略下的带宽平均降低 35.96%。该阶段 C_{out} 较小, 在多核间沿通道维度优先展开的部署模式下, 模型权重可完整装载至片上缓存, 从而逼近理论带宽下限。随着网络深度增加, 片上缓存容量已不足以完整容

纳任何类型的数据, 中间层由此呈现出两类特征分布: 在较浅的中层 (Layer7~11), OST 策略较 OSC 实现 15.98% 的平均带宽降低。此阶段 $H'W'$ 显著高于 C_{out} , OST 通过将空间维度更充分地映射至片上, 实现更高的权重复用效率, 进而显著减少 DDR 访存次数。而在较深的中层 (Layer12~16), OSC 策略相较于 OST 平均带宽下降 7.27%。此时 C_{out} 逐渐占据主导地位, OSC 策略利用输出通道方向的高并行度优化激活值的复用行为, 有效降低片外访存压力。进入模型深层 (Layer17~21), OST 策略再次显现优势, 较 OSC 带宽平均降低 2.32%。该阶段空间分辨率急剧下降, 沿空间分辨率维度展开的多核部署方式可使激活值完整装载至片上缓存, 带宽性能再度逼近理论下限。

上述分析表明, 对于以 CNN 为代表的深度层次化网络, 其层间参数特征随深度递进而呈现显著演化。OSC 与 OST 两种并行策略在各层的带宽性能优劣并非固定, 而由当前层的张量维度配置与片上缓存容量共同约束、动态决定。随着现代神经网络模型层数持续加深, 不同层在最优多核并行策略选择上的差异愈加显著, 其整体访存带宽的影响权重亦随之提升。由此, 各层对并行策略的敏感度明显增强, 部署策略依据层特征动态调整的必要性 with 优化效益日益凸显。针对上述深度层次化结构, 本文提出一种自适应部署策略, 该策略依据各层计算特征, 自适应调控张量并行切分维度与数据流映射方式。相关具体实现机制将在下一小节算子层映射动态优化框架中予以系统阐述。

表 2 ViT-base 部分层参数表

Table 2 ViT-base Partial Layer Parameters Table

模型层	算子	尺寸	
Layer1	GEMM_Bias	输入	(197, 1, 768)
		权重	(768, 768, 1, 1)
		输出	(197, 1, 768)
Layer4	GEMM_MulConstant	输入	(197, 1, 64)
		权重	(197, 64, 1, 1)
		输出	(197, 1, 197)
Layer5	SoftMax	输入	(197, 1, 197)
		输出	(197, 1, 197)
		输入	(197, 1, 197)
Layer6	GEMM	权重	(64, 197, 1, 1)
		输出	(197, 1, 64)
		输入	(197, 1, 768)
Layer8	LayerNorm	输出	(197, 1, 768)
		输入	(197, 1, 768)
		权重	(3072, 768, 1, 1)
Layer9	GEMM_Bias_Gelu	输出	(197, 1, 3072)
		输入	(197, 1, 3072)
		权重	(197, 1, 3072)

权重	(768, 3072, 1, 1)
输出	(197, 1, 768)

表 3 ResNet-18 部分层参数表
Table 3 ResNet-18 Partial Layer Parameters Table

模型层	算子		尺寸
浅层	Layer0	Conv_Bias_Rel	激活 (224, 224, 16)
			权重 (64, 16, 7, 7)
			输出 (112, 112, 64)
	Layer2	Conv_Bias_Rel	激活 (56, 56, 64)
			权重 (64, 64, 3, 3)
			输出 (56, 56, 64)
	Layer6	Conv_Bias_Rel	激活 (56, 56, 64)
			权重 (128, 64, 3, 3)
			输出 (28, 28, 128)
	Layer7	Conv_Bias	激活 (56, 56, 64)
			权重 (128, 64, 1, 1)
			输出 (28, 28, 128)
中间层	Layer8	Conv_Bias_Res_Rel	激活 (28, 28, 128)
			权重 (128, 128, 3, 3)
			输出 (28, 28, 128)
	Layer11	Conv_Bias_Rel	激活 (28, 28, 128)
			权重 (256, 128, 3, 3)
			输出 (14, 14, 256)
	Layer12	Conv_Bias	激活 (28, 28, 128)
			权重 (256, 128, 1, 1)
			输出 (14, 14, 256)
	Layer13	Conv_Bias_Res_Rel	激活 (14, 14, 256)
			权重 (256, 256, 3, 3)
			输出 (14, 14, 256)
深层	Layer17	Conv_Bias	激活 (14, 14, 256)
			权重 (512, 256, 1, 1)
			输出 (7, 7, 512)
	Layer18	Conv_Bias_Res_Rel	激活 (7, 7, 512)
			权重 (512, 512, 3, 3)
			输出 (7, 7, 512)
	Layer20	GlobalAveragePool	输入 (7, 7, 512)
			输出 (1, 1, 512)
			激活 (1, 1, 512)
	Layer21	GEMM_Bias	权重 (32, 512, 1, 1)
			输出 (1, 1, 32)

3.4 算子层动态映射优化框架

在传统基于固定算力的 ASIC 芯片的硬件映射研

究策略中, 由于算力与缓存比例已被绑定, 对于一个算子层的计算时间 (所需算力/硬件算力) 是恒定的。相关研究主要聚焦于不同映射方法对数据通信量的优化

效果,其核心目标是最小化片外访存开销。相比之下,本文基于上述多核可重构计算-缓存架构,提出算子层动态映射优化框架。将硬件映射策略的优化目标拓展为多维协同优化问题,通过在不同计算阶段动态配置缓存资源与数据通路,该架构使得实际有效算力与可用带宽能够更灵活地适配计算任务的实时需求。在神经网络推理场景中,该机制可动态调整架构部署目标层任务的算力-带宽比使其最大限度逼近每层网络所需的算术强度,从而在运行时最大限度地实现计算与数据搬运的重叠执行,实现张量计算的加速。

一个张量计算层所需的理论算力为:

$$OPs = C_{out} H_{out} W_{out} (2C_{in} K_h K_w - 1) \quad (14)$$

针对输入数据带宽,依据 3.3 节对带宽的讨论,可将其负载模式归纳为三种典型情形:权重完全可缓存 (fully cacheable weights, FCW)、激活值完全可缓存 (fully cacheable activations, FCA) 以及两者均无法完全缓存而导致数据需逐块重复加载 (reload per tile, RPT)。在 FCW 情形下,权重一次性加载完成后,带宽负载优化域仅涉及激活值的读取。相应地,在 FCA 情形下,带宽负载优化域仅涉及权重的读取。而在 RPT 情形下,则需同时考虑权重与激活值的重复加载;若采用多 DMA 并行加载架构,总带宽需求取两者中的较大值。基于上述分析,负载优化域的带宽可表示为:

$$BW = \begin{cases} C_{in} H W b_{act}, & FCW \\ C_{out} C_{in} K_h K_w b_{act}, & FCA \\ (\alpha C_{in} H W + \beta C_{out} C_{in} K_h K_w) b_{act}, & RPT \end{cases} \quad (15)$$

假设硬件单核可提供的峰值算力为 T / clk , 其有效带宽为 B / clk 。则硬件的最优映射策略可化为式(16)所示优化问题,其中 n 为面向任务-资源特征所激活的计算核数, m 为动态调整计算核数所带来的缓存收益, $m \times n = N$, N 为总核数。

$$\begin{aligned} n, C_{bog}, h', w' = \arg \min_{C_{bog}, h', w'} & (\max(\frac{OPs}{nT}, \frac{BW}{B})) \\ \text{s.t. } C_{bog} h' w' & \leq \frac{\min\{m S_{cache}, C_{out} H W'\}}{P_o} \\ h w & \leq \frac{\min\{m I_{cache}, C_{in} H W\}}{P_i} \\ C_{bog} & \leq \frac{\min\{m W_{cache}, C_{out} C_{in} K_h K_w\}}{P_o P_i K_h K_w} \\ n, C_{bog}, h', w' & \in \mathbb{Z}_0^+ \end{aligned} \quad (16)$$

图 10 展示了算子层动态映射优化框架。在可重构架构下,数据完全缓存并不必然带来整体执行时间的最优解。适度重复加载数据有时可换取更高的计算并行度或更均衡的硬件利用率。因此,本文将 FCW、FCA 与 RPT 三种缓存情形视为同等重要的候选解空间,在优化过程中予以并行评估。同时,针对 3.3 节讨论的

数据部署策略带来的带宽差异,优化框架在求解优化目标前需依据所选展开策略动态调整缓存容量约束,并分别针对 OSC 与 OST 搜索最优分块形状与硬件配置,最终选取使整体执行时间最小的数据流策略、硬件参数及分块尺寸。若在不同数据流或缓存情形下均能实现相同的理论最小执行时间,则表明延迟瓶颈已转移至计算单元本身。此时,优化重心应从单纯追求速度转向在同等性能下追求更高能效,选取总数据通信量最小的配置,以最小化片外数据搬运带来的动态能耗。

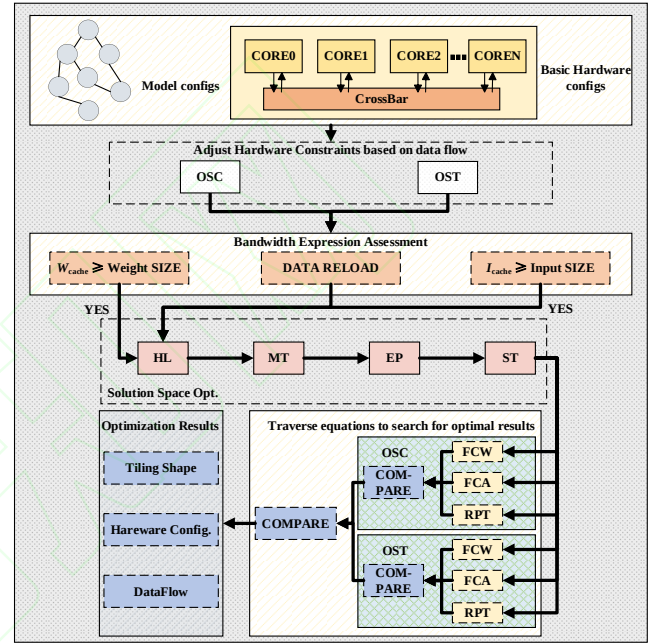


图 10 算子层动态映射优化框架

Fig.10 Operator Layer Dynamic Mapping Optimization Framework

鉴于上述优化问题中未知参数的数量超过了可建立的独立方程数量,该问题本质上构成一个欠定系统,无法直接解析方程求解该优化目标,只能通过枚举遍历其解空间求出最优解。尽管引入片上缓存容量等硬件约束(hardware limitations, HL)能显著压缩解空间规模,但对于高实时性、低延迟要求的应用场景,基于枚举的搜索方法仍面临组合爆炸问题,其计算开销难以满足快速编译与调度的需求。为此,本文提出了一套结构化解空间约简策略,系统整合了以下三项原则:

最大分块原则 (max tiling, MT): 要求在硬件资源约束下,数据块尺寸应尽可能最大化,以提升片上数据复用率,降低片外带宽压力。

整分原则 (Evenly Partitioning, EP): 规定子块尺寸必须为原始维度的整数因子,以确保计算核间负载均衡,简化地址映射与控制逻辑。

方形分块原则 (Square Tiling, ST): 基于数据访问的局部性特征,使各维度分块尺寸趋于均衡,以最小

化边界填充与通信开销。

上述原则共同构成一组强约束条件,系统性地剔除了大量低效或不可行的映射方案。图 11 以 ResNet-18 模型为例,逐级引入上述原则后,解空间规模较仅

采用硬件约束时分别平均缩小 77.64% (MT)、72.75% (EP) 和 39.81% (ST),显著降低了编译器的搜索复杂度。

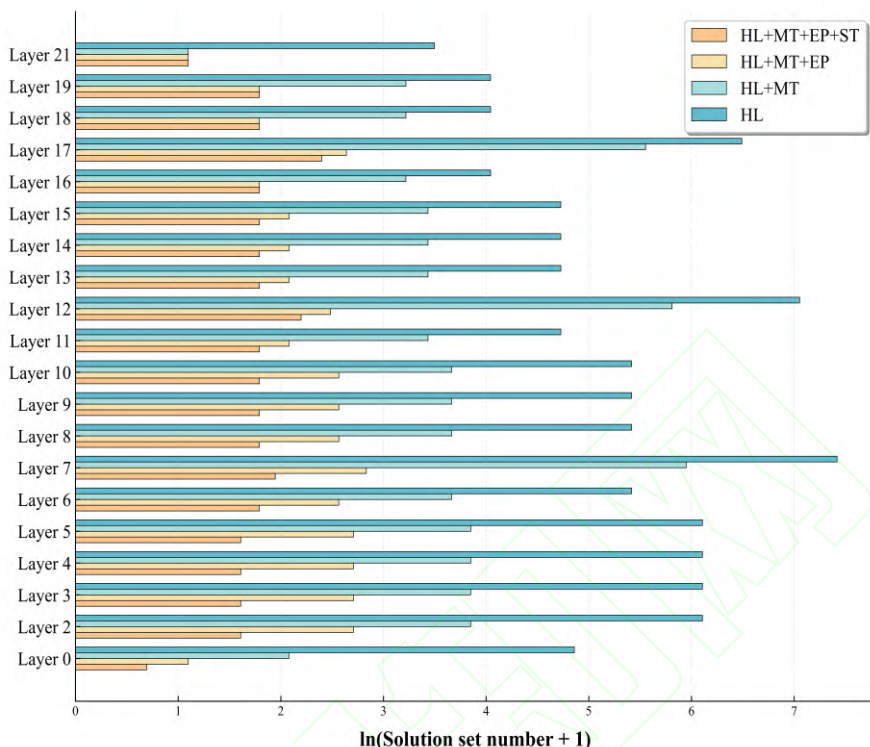


图 11 ResNet-18 模型解空间优化

Fig.11 ResNet-18 Deconvolution Space Optimization

得益于算子层动态映射优化框架能够根据模型具体结构和运行时需求自适应调整参数的特性,该框架展现出卓越的普适性。无论面向卷积神经网络、Transformer 还是其他新兴的深度学习模型架构,其均能通过动态适配各算子独特的数据访问模式,有效识别并应用最优的分块策略,数据流以及硬件配置。从而在不同模型上实现显著且稳健的带宽需求降低效果。更重要的是,该优化框架求解出的最优解集及其反映出的数据重用模式,为片上存储子系统的设计提供了宝贵的量化依据。通过分析不同模型、不同算子下资源约束与最优分块参数的关系,可以揭示出对缓存容量、带宽等关键架构资源的需求规律,指导设计者在存储资源分配、带宽配置等方面做出更精准的权衡决策,提升整体硬件架构的运行效率。

4 实验结果与分析

4.1 实验环境

基于上述带宽优化策略,本研究设计一款如图 12 所示的神经网络处理单元 (Neural Processing Unit, NPU) 架构。为全面评估该架构的性能表现与优化效果,实验依托于依据该设计制成的实物芯片展开。NPU 主频设置为 500MHz, DDR 主频设置为 466MHz。

本文所采用的 NPU 架构共由 PEA、VCU、LOAD、STORE、SYNCHRONIZE、INSN_FIFO_WRAPPER 以及各数据片上 SRAM 组成。DISPATCH 模块作为指令数据从片外存储单元加载到片内缓存 FIFO 单元的传输模块,负责指令数据的调度与传输。SYNCHRONIZE 模块作为输入输出数据以及计算单元的同步控制模块,确保数据传输与计算操作的同步协调。LOAD、STORE 模块负责数据的加载与存储。PEA 和 VCU 模块作为架构中的计算单元,其中 PEA 模块负责卷积运算,VCU (vector compute unit, VCU) 模块负责向量运算 (如加 bias、激活函数等操作),共同完成 AI 推理过程中的核心计算任务。SRAM 模块为片内各数据的缓存单元。

为不失一般性,架构中将 NPU 划分为四个处理簇,每个簇包含两个计算核,NPU 簇内主核支持 SRAM 交叉访存。簇内主核通过对 SRAM 的动态映射与交叉访问,可显著降低数据带宽需求与传输延迟,提高数据处理的整体吞吐量。

架构中单计算核缓存资源如表 4 所示,所有资源都采用多重缓存设计以实现模块流水并行。

表4 单核心缓存资源

Table 4 Intra-Core Cache Resource Allocation

缓存类型	位宽×深度	BANK 数量
------	-------	---------

Ifmap_ram	256 × 512	2
Weight_ram	256 × 2048	1
Psum_ram	1024 × 1024	2

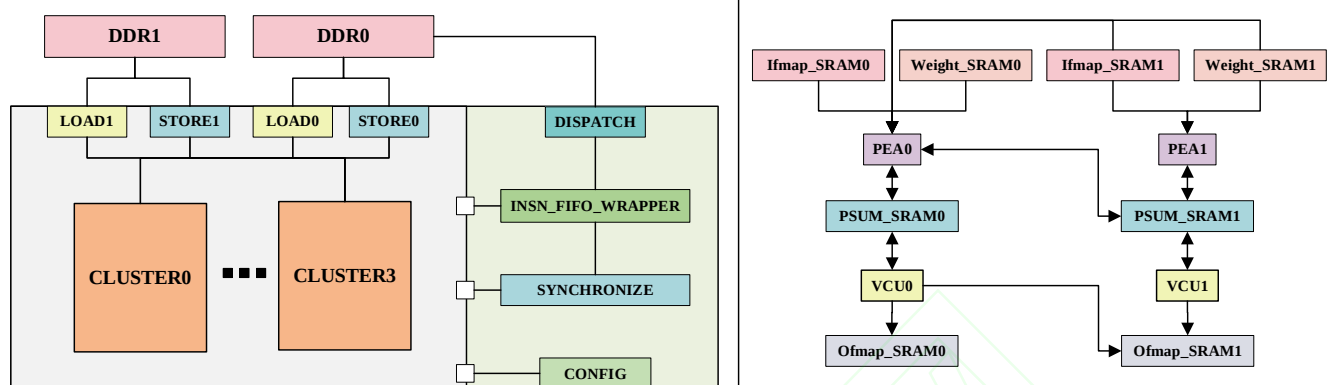


图12 NPU架构(左)与簇内架构(右)

Fig.12 NPU Architecture (Left) and Intra-Cluster Architecture (Right)

4.2 单算子优化效果评估

为系统评估所提出的多核计算-缓存可重构架构及其配套的算子层动态映射优化框架的有效性, 本文设计四组对照实验。实验基于上述同一基准 NPU 硬件平台, 其中传统不可重构架构通过禁用重构机制实现, 以模拟当前主流的静态资源绑定设计。各组实验配置如下:

第一组: 硬件采用传统不可重构架构, 软件映射策略采用算子层动态映射优化框架。该组旨在衡量映射优化框架本身在静态硬件上的优化潜力。

第二/三组: 硬件启用所提可重构架构, 软件映射策略采用广度优先算法 (Breadth First) /深度优先算法 (Depth First)。该组用于检验可重构硬件在传统搜索策略下的性能表现。

第四组: 硬件启用可重构架构, 软件映射策略采用算子层动态映射优化框架。该组综合了所提硬件与软件协同设计, 用于全面验证其相较于静态架构及传统映射策略的优势。

实验选取 3 种主流的 AI 模型的部分算子层, 分别是卷积神经网络模型 Resnet-50、YOLOv5s 和典型 Transformer 架构模型 ViT-Base。单算子参数如表 5 所示。实验测试单算子的推理耗时和算力释放效率, 可有效验证本文算法在缓解带宽压力、提升数据复用效率

方面的优化效果。由于数据分块采用均匀划分, 本文以计算与数据加载重叠执行阶段中, 单一数据块的计算时间与下一数据块加载时间的比值, 来衡量算子的算力释放效率。

单算子推理耗时测试结果如图 13 (a) 所示, 相较于第一组 (静态硬件+算子层动态映射优化框架) 基线, 所提多核计算-缓存可重构架构将各类单算子推理耗时降低 5.76%~41.34%。这一性能提升源于可重构架构对映射可行域的根本性扩展, 使硬件能更精细地适配算子级计算特征, 从而优化计算-通信重叠效率并显著削减片外访存开销。相较于采用广度优先或深度优先固定映射策略的第二/三组, 本文方案实现 18.16%~69.45% 的耗时降低, 不仅验证了所提优化框架在端到端推理加速中的有效性, 更揭示了神经网络中算子张量形状的多样性对最优存储映射策略的动态影响——传统单一路径搜索策略难以应对该变化, 而本文框架能够在给定存储约束下实时搜索并适配各算子的最优数据布局、分块方案与架构配置, 赋予硬件对多样化模型乃至单一模型内异构算子特征的灵活适应能力。特别对于带宽受限型算子, 该机制在片上实现更高的数据复用效率, 显著降低片外访存流量, 凸显了其在性能与架构适应性方面的综合优势。

结合图 13 (b) 所示的算力释放效率分析, 对于在

充分激活算力核下算力利用率已较高的层(如 resnet50 layer4),进一步优化所获得的加速收益相对有限。结合图 13(c)给出的 VCU 计算耗时则表明,由于 PEA 与 VCU 之间采用紧耦合的数据通路绑定,架构配置重构会引起向量与逐元素计算耗时的增加。此外,分块尺寸调整导致分块数量减少时,部分 VCU 单元处于空闲状态,算力未能充分释放,进一步增加了 VCU 端的计算开销。上述 VCU 计算耗时的增长,是导致部分算子层(如 resnet50 layer27)加速效果不显著的重要原因。在下一代芯片的设计中,将尝试把张量计算与向量计算通路解耦,以充分释放 VCU 算力,解决上述性能瓶颈。

4.3 模型优化结果与分析

本文设计的 NPU 架构中的张量计算单元采用输入通道与输出通道并行计算的结构,在 500 MHz 工作频率下,其峰值算力在 Int8 精度下达到 8 TOPS,在 FP16 精度下为 4 TFLOPS。为系统验证本文所提软硬件协同优化方法的综合性能优势,本文选取搭载于 Atlas 200 DK 开发者套件的华为昇腾 310 芯片与英伟达 GeForce RTX 2060 GPU 作为对照平台,在 YOLOv5s、ResNet-50 以及 ViT-Base 三类模型上进行推理耗时对比实验,模型数据精度皆为 FP16,以实测延迟和能效比作为核心评估指标。为保证公平性,推理耗时仅测试

算力芯片运行时间,CPU 编译时间不包含在推理耗时之内。平台相关参数与模型推理耗时结果如表 6 所示。

YOLOv5s 模型测试实验结果表明,本芯片在实际推理任务中展现出超越昇腾 310 的能效优势,更大幅超越 GeForce RTX 2060 GPU,达到后者的 20 倍以上。这一结果有力地印证了本文在数据流组织、存储访存管理及动态映射机制等方面所采用的软硬件协同优化方法的有效性,为面向边缘计算场景的高能效 AI 芯片设计提供了具有参考价值的技术范本。

本文在 ResNet-50 与 ViT-Base 模型上的推理耗时相较于昇腾 310 与 2060 GPU 不具有优势,其根本原因在于当前架构缺乏 PEA-VCU-PEA 循环数据通路,难以支持多层张量计算在片上的连续驻留与图级优化,迫使层间数据经 DDR 中转,从而抵消了单算子层的带宽优化收益。相比之下,对比平台借助图级算子融合技术显著降低了片外访存需求。图级优化在大空间分辨率场景下的增益有限,这也解释了本文所提软硬件协同优化方法在 YOLOv5s 模型测试中表现更为优异的原因。上述局限并不削弱本文优化方法的适用性,本文提出的优化方案与图级优化互补,二者结合将为本文下一代芯片与软件设计提供更为完备的性能优化空间。

表 5 单算子参数表

Table 5 Single-operator Parameter List

模型层	算子	尺寸
resnet50 layer4	Conv_Bias_Relul_Conversion	输入 (56, 56, 64)
		权重 (64, 64, 3, 3)
resnet50 layer8	Conv_Bias_Res_Relul_Conversion	输入 (56, 56, 64)
		权重 (256, 64, 1, 1)
resnet50 layer14	Conv_Bias_Relul_Conversion	输入 (56, 56, 128)
		权重 (128, 128, 3, 3)
resnet50 layer27	Conv_Bias_Relul_Conversion	输入 (28, 28, 256)
		权重 (256, 256, 3, 3)
yolov5s layer17	Conv_Bias_Sigmoid_Resmul_Conversion	输入 (80, 80, 128)
		权重 (128, 256, 3, 3)
yolov5s layer28	Conv_Bias_Sigmoid_Resmul_Conversion	输入 (40, 40, 256)
		权重 (256, 256, 3, 3)
yolov5s layer56	Conv_Bias_Sigmoid_Resmul_Conversion	输入 (80, 80, 64)
		权重 (64, 64, 3, 3)

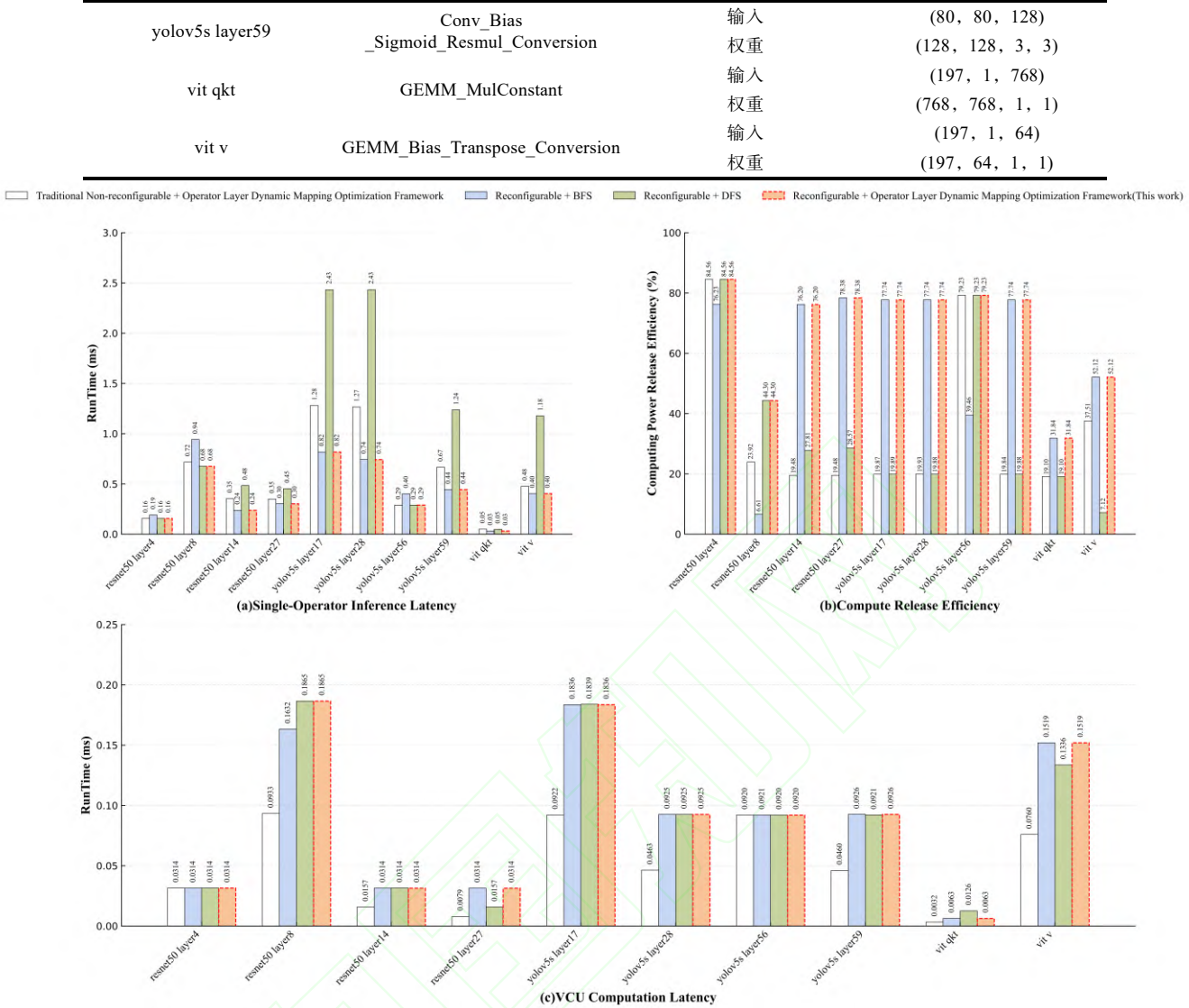


图 13 (a)单算子推理耗时、(b)算力释放效率与(c)VCU 计算耗时测试结果

Fig.13 Test Results of (a) Single-Operator Inference Latency, (b) Compute Release Efficiency, and (c) VCU Computation Latency

表 6 平台参数与模型测试结果

Table 6 Platform Parameters and Model Test Results

	RTX 2060	昇腾 310 ^[38]	本工作
工艺	12nm	-	28nm
CPU	Intel Core i7-10875H@2.30 GHz四核	ARM Cortex-A55@1.6 GHz四核	Intel Gold G6405@4.10GHz
算力	8TFLOPS@FP16	4TFLOPS@FP16	4TFLOPS@FP16
功耗	160W	9W	10W
软件	CUDA	ATC8.2	算子层动态映射优化框架
模型	尺寸		
YOLOv5s	640 × 640 × 3		
推理耗时(ms)	35.06	31.78	27.64
ResNet-50	224 × 224 × 3		
	11.99	7.03	13.86
ViT-base	197 × 1 × 768		
	53.99	41.62	52.31
YOLOv5s	640 × 640 × 3		
能效比(FPS/W)	0.18	3.50	3.62
ResNet-50	224 × 224 × 3		
	0.52	15.81	7.22
ViT-base	197 × 1 × 768		
	0.12	2.67	1.91

5 结论

在当前计算加速架构普遍采用计算单元与累加器缓存紧密耦合设计的背景下，针对权重与激活值重复

访存所导致的系统能效瓶颈，本文提出了一种面向多核加速器架构的软硬件协同带宽优化方案。该方案基于多核协同广播传输机制，通过片上网络实现激活值与权重的核间共享，将分块维度从单核扩展至多核协

同范围,从数据流层面显著提升复用效率。其次,将算力与缓存资源解耦,提出可重构计算-缓存架构,使系统能够在任务粒度动态调整算力-缓存配比,有效适配多样化网络层的算术强度需求。在此基础上,基于典型模型的实验分析表明,最优并行策略随张量维度演化而动态变化,论证了自适应部署的必要性。进而构建算子层动态映射优化框架,将数据流、分块策略与硬件配置纳入协同优化范畴,并引入最大分块、整分与方形分块三项结构化解空间约简原则,在保证优化质量的前提下显著降低搜索复杂度。基于上述芯片实测结果表明,所设计的加速器在 YOLOv5s 推理任务中能表现优于昇腾 310,并达到 GeForce RTX 2060 GPU 的 20 倍以上,验证了本文方法在架构优化和实际部署中的有效性。需要指出的是,当前工作尚未充分考虑模型单算子中向量计算部分对推理延迟的影响,且多核协同广播机制在规模扩展时可能引发核间互连拥塞及关键路径时序收敛性恶化。未来研究将重点探索张量计算与向量计算对系统带宽的协同影响机制,并研究面向 SRAM 间高效数据调度的互连架构优化方法,在维持高计算效率的同时进一步提升系统的可扩展性与时序鲁棒性。

参考文献:

- [1] LI G H, MANDAL S K, OGRAS U Y, et al. FLASH: Fast Neural Architecture Search with Hardware Optimization[J]. ACM Transactions on Embedded Computing Systems, 2021, 20(5): 1–26.
- [2] 潘晓英, 曹园, 贾蓉, 等. 神经网络架构搜索发展综述[J]. 西安邮电大学学报, 2022, 27(4): 43–63.
PAN X Y, CAO Y, JIA R, et al. Overview of neural network architecture search development[J]. Journal of Xi'an University of Posts and Telecommunications, 2022, 27(4): 43–63.
- [3] DAHL G E, YU D, DENG L, et al. Context-Dependent Pre-Trained Deep Neural Networks for Large-Vocabulary Speech Recognition[J]. IEEE Transactions on Audio, Speech, and Language Processing, 2012, 20(1): 30–42.
- [4] 张坤宁, 赵烁, 孙庆斌, 等. 基于 FPGA 的多核可扩展卷积加速器设计[J]. 计算机工程与设计, 2021, 42(6): 1592–1598.
ZHANG K N, ZHAO S, SUN Q B, et al. Design of CNN accelerator with multi-core based on FPGA[J]. Computer Engineering and Design, 2021, 42(6): 1592–1598.
- [5] SATESSAN A, SINHA S, K G S, et al. A Survey of Algorithmic and Hardware Optimization Techniques for Vision Convolutional Neural Networks on FPGAs[J]. Neural Processing Letters, 2021, 53(3): 2331–2377.
- [6] YIN S Y, OUYANG P, TANG S B, et al. A High Energy Efficient Reconfigurable Hybrid Neural Network Processor for Deep Learning Applications[J]. IEEE Journal of Solid-State Circuits, 2018, 53(4): 968–982.
- [7] SHAN R, GAO X, FENG Y N, et al. Design and implementation of near-memory computing array architecture based on shared buffer[J]. High Technology Letters, 2022, 28(4): 345–353.
- [8] KULKARNI U, HOSAMANI A S, MASUR A S, et al. A Survey on Quantization Methods for Optimization of Deep Neural Networks[C]//Proc of the 2022 International Conference on Automation, Computing and Renewable Systems. Piscataway, NJ: IEEE, 2022: 827–834.
- [9] JOUPPI N P, YOUNG C, PATIL N, et al. In-Datcenter Performance Analysis of a Tensor Processing Unit[C]//Proc of the 44th International Symposium on Computer Architecture. Toronto: ACM, 2017: 1–12.
- [10] CHEN T S, DU Z D, SUN N H, et al. DianNao: a small-footprint high-throughput accelerator for ubiquitous machine-learning[J]. SIGARCH Computer Architecture News, 2014, 42(1): 269–284.
- [11] CHEN Y H, EMER J, SZE V. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks[C]//Proc of the 43rd International Symposium on Computer Architecture. Seoul: IEEE, 2016: 367–379.
- [12] HAN S, LIU X Y, MAO H Z, et al. EIE: Efficient Inference Engine on Compressed Deep Neural Network[C]//Proc of the 43rd International Symposium on Computer Architecture. Seoul: IEEE, 2016: 243–254.
- [13] SENGUPTA J, KUBENDRAN R, NEFTCI E, et al. High-Speed, Real-Time, Spike-Based Object Tracking and Path Prediction on Google Edge TPU[C]//Proc of the 2020 IEEE International Conference on Artificial Intelligence Circuits and Systems. Genova: IEEE, 2020: 134–135.
- [14] HAN S, MAO H Z, DALLY W J. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding[C]//Proc of the 4th International Conference on Learning Representations. San Juan: ICLR, 2016.
- [15] IGNJATOVIĆ D, BAILEY D W, BAJIĆ L. The Wormhole

- AI Training Processor[C]//Proc of the IEEE International Solid-State Circuits Conference. San Francisco: IEEE, 2022: 356–358.
- [16] GAO M Y, YANG X, PU J, et al. TANGRAM: Optimized Coarse-Grained Dataflow for Scalable NN Accelerators[C]//Proc of the International Conference on Architectural Support for Programming Languages and Operating Systems. Providence: ACM, 2019: 807–820.
- [17] TAN Z H, CAI H Y, DONG R P, et al. NN-Baton: DNN Workload Orchestration and Chiplet Granularity Exploration for Multichip Accelerators[C]//Proc of the IEEE International Symposium on Computer Architecture. Valencia: IEEE, 2021: 1013–1026.
- [18] ZHENG S X, ZHANG X J, LIU L B, et al. Atomic Dataflow based Graph-Level Workload Orchestration for Scalable DNN Accelerators[C] // Proc of the IEEE International Symposium on High-Performance Computer Architecture. Seoul: IEEE, 2022: 475–489.
- [19] HOEFLER T, ALISTARH D, BEN-NUN T, et al. Sparsity in deep learning: pruning and growth for efficient inference and training in neural networks[J]. Journal of Machine Learning Research, 2021, 22(1): 10882–11005.
- [20] ZHU M H, ZHANG T, GU Z Y, et al. Sparse Tensor Core: Algorithm and Hardware Co-Design for Vector-wise Sparse Neural Networks on Modern GPUs[C]//Proc of the 52nd International Symposium on Microarchitecture. Columbus: ACM/IEEE, 2019: 359–371.
- [21] MOON S, LI M, CHEN G, et al. T-REX: A 68-to-567 μ s/Token 0.41-to-3.95 μ J/Token Transformer Accelerator with Reduced External Memory Access and Enhanced Hardware Utilization in 16nm FinFET[C]//Proc of the IEEE International Solid-State Circuits Conference. San Francisco: IEEE, 2025: 406–408.
- [22] DU J C, JIN R R, HUANG W W, et al. Optimize Quantization for Large Language Models via Progressive Training[C]//Proc of the ACM Web Conference 2025 Companion. New York: ACM, 2025: 2474–2483.
- [23] FRANTAR E, ASHKBOOS S, HOEFLER T, et al. OPTQ: Accurate Quantization for Generative Pre-trained Transformers[C]//Proc of the 11th International Conference on Learning Representations. New York: ACM, 2023.
- [24] DETTMERS T, SVIRSCHEVSKI R, EGIAZARIAN V, et al. SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression [C]//Proc of the 12th International Conference on Learning Representations. New York: ACM, 2024.
- [25] AHN J, HONG S, YOO S, et al. A scalable processing-in-memory accelerator for parallel graph processing[C]//Proc of the 42nd International Symposium on Computer Architecture. New York: ACM, 2015: 105–117.
- [26] AHN J, YOO S, MUTLU O, et al. PIM-Enabled Instructions: A Low-Overhead, Locality-Aware Processing-in-Memory Architecture[C]//Proc of the 42nd International Symposium on Computer Architecture. New York: ACM, 2015: 336–348.
- [27] LOCKERMAN E, FELDMANN A, BAKHSHALIPOUR M, et al. Livia: Data-Centric Computing Throughout the Memory Hierarchy[C]//Proc of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems. New York: ACM, 2020: 417–433.
- [28] SANGAIAH K, LUI M, KUTTAPPA R, et al. SnackNoC: Processing in the Communication Layer[C]//Proc of the IEEE International Symposium on High-Performance Computer Architecture. Piscataway, NJ: IEEE, 2020: 461–473.
- [29] SCHWEDOCK B C, YOOVIDHYA P, SEIBERT J, et al. tākō: a polymorphic cache hierarchy for general-purpose optimization of data movement[C]//Proc of the 49th International Symposium on Computer Architecture. New York: ACM, 2022: 42–58.
- [30] WANG Z R, LIU C, ARORA A, et al. Infinity Stream: Portable and Programmer-Friendly In-/Near-Memory Fusion[C]//Proc of the 28th International Conference on Architectural Support for Programming Languages and Operating Systems. New York: ACM, 2023, 3: 359 – 375.
- [31] WANG Z R, WENG J, LIU S H, et al. Near-Stream Computing: General and Transparent Near-Cache Acceleration[C]//Proc of the IEEE International Symposium on High-Performance Computer Architecture. Piscataway, NJ: IEEE, 2022: 331–345.
- [32] CHATARASI P, KWON H, PARASHAR A, et al. Marvel: A Data-Centric Approach for Mapping Deep Learning Operators on Spatial Accelerators[J]. ACM Transactions on Architecture and Code Optimization, 2022, 19(1): 1–26.
- [33] HEGDE K, TSAI P A, HUANG S, et al. Mind mappings:

- enabling efficient algorithm-accelerator mapping space search[C]//Proc of the 26th International Conference on Architectural Support for Programming Languages and Operating Systems. New York: ACM, 2021: 943–958.
- [34] KAO S C, KRISHNA T. GAMMA: Automating the HW Mapping of DNN Models on Accelerators via Genetic Algorithm[C]//Proc of the IEEE/ACM International Conference on Computer-Aided Design. Piscataway, NJ: IEEE, 2020: 1–9.
- [35] KWON H, CHATARASI P, PELLAUER M, et al. Understanding Reuse, Performance, and Hardware Cost of DNN Dataflow: A Data-Centric Approach[C]//Proc of the IEEE/ACM International Symposium on Microarchitecture. New York: ACM, 2019: 754–768.
- [36] PARASHAR A, RAINA P, SHAO Y S, et al. Timeloop: A Systematic Approach to DNN Accelerator Evaluation[C]//Proc of the IEEE International Symposium on Performance Analysis of Systems and Software. Piscataway, NJ: IEEE, 2019: 304–315.
- [37] TAN Z H, ZHU Z J, MA K S. Cocco: Hardware-Mapping Co-Exploration towards Memory Capacity-Communication Optimization[C]//Proc of the 29th International Conference on Architectural Support for Programming Languages and Operating Systems. New York: ACM, 2024, 1: 69–84.
- [38] LIAO H, TU J J, XIA J, et al. Ascend: a Scalable and Unified Architecture for Ubiquitous Deep Neural Network Computing: Industry Track Paper[C]//Proc of the 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA). 2021: 789–801. DOI: 10.1109/HPCA51647.2021.00071.